AI ASSISTANT CODING
ASSIGNMENT - 20.3

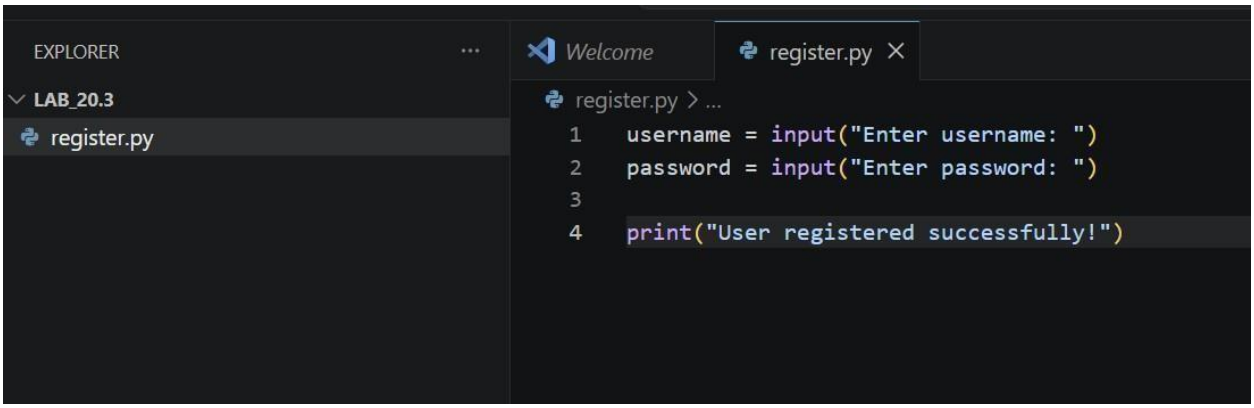
Name: P.Ram charan
Ht.no: 2303A51298
Batch: 05

Task1: Password Strength & Validation Check

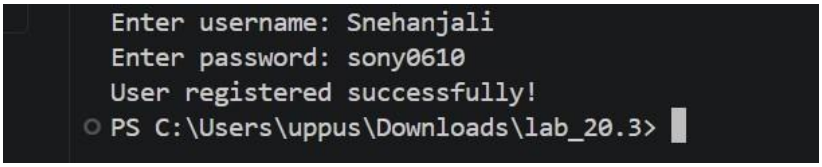
Prompt: Generate Python user registration program

Code:

AI-Generated Code:



Output:



- The program accepts passwords without any validation checks.
- Weak passwords are allowed, making the system insecure.

Corrected Code:

```python
import re
def is_strong_password(password):
    if len(password) < 8:
        print("❌ Minimum 8 characters required")
        return False
    if not re.search(r"[A-Z]", password):
        print("❌ Must include uppercase letter")
        return False
    if not re.search(r"[a-z]", password):
        print("❌ Must include lowercase letter")
        return False
    if not re.search(r"[0-9]", password):
        print("❌ Must include number")
        return False
    if not re.search(r"[!@#$%^&*]", password):
        print("❌ Must include special character")
        return False
    return True

username = input("Enter username: ")
password = input("Enter password: ")

if is_strong_password(password):
    print("✅ User registered successfully!")
else:
    print("❌ Weak password! Try again.")
```

Output:

```
PS C:\Users\uppus\Downloads\lab_20.3> python register.py
Enter username: Snehanjali
Enter password: Sony@0610
✅ User registered successfully!
PS C:\Users\uppus\Downloads\lab_20.3>
```

Explanation:

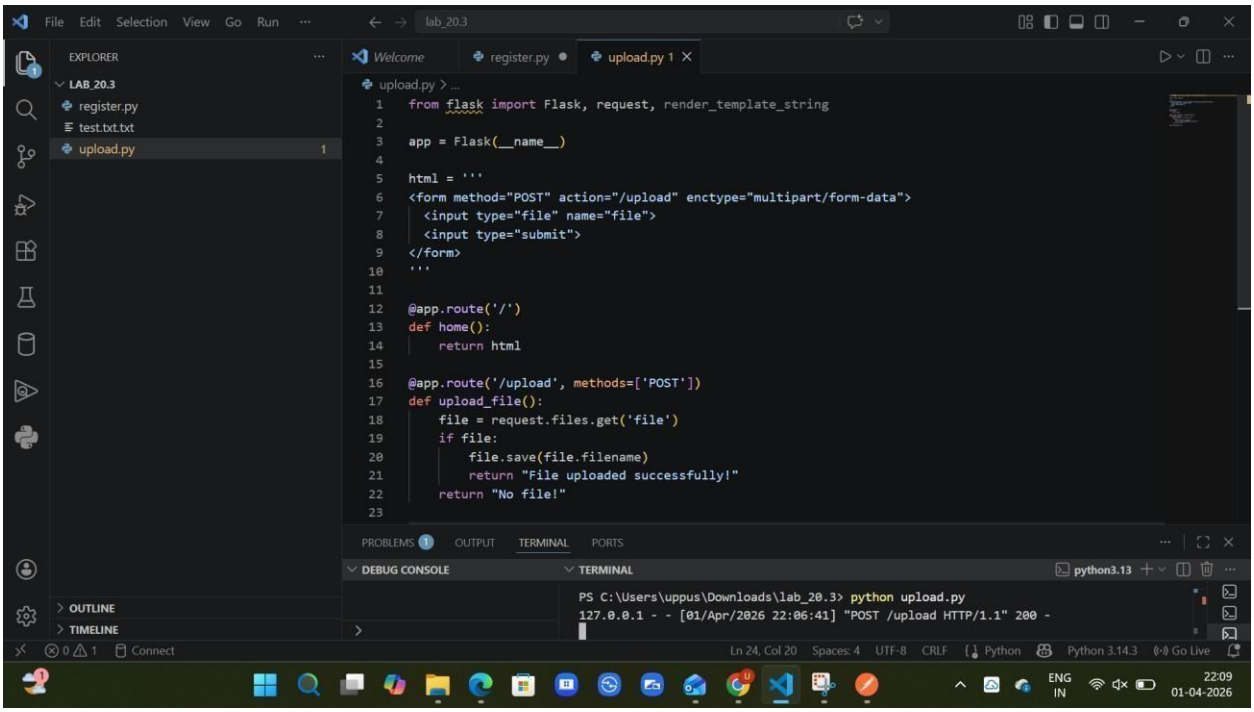
The initial program does not check the strength of the password. It accepts any password entered by the user, including weak ones. This makes the system insecure and vulnerable to attacks.

Task2: Insecure File Upload Handling

Prompt:

Generate a Flask-based file upload program that allows users to upload files.

Code:Ai Generated Code

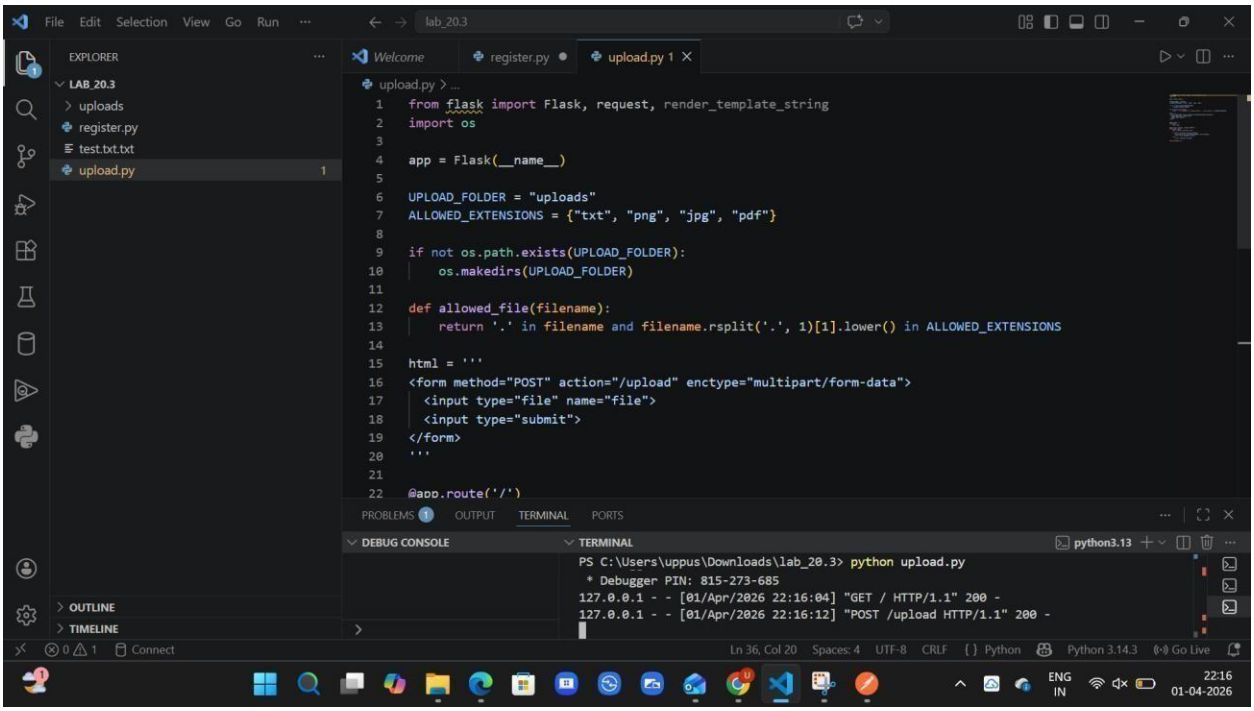


Output:

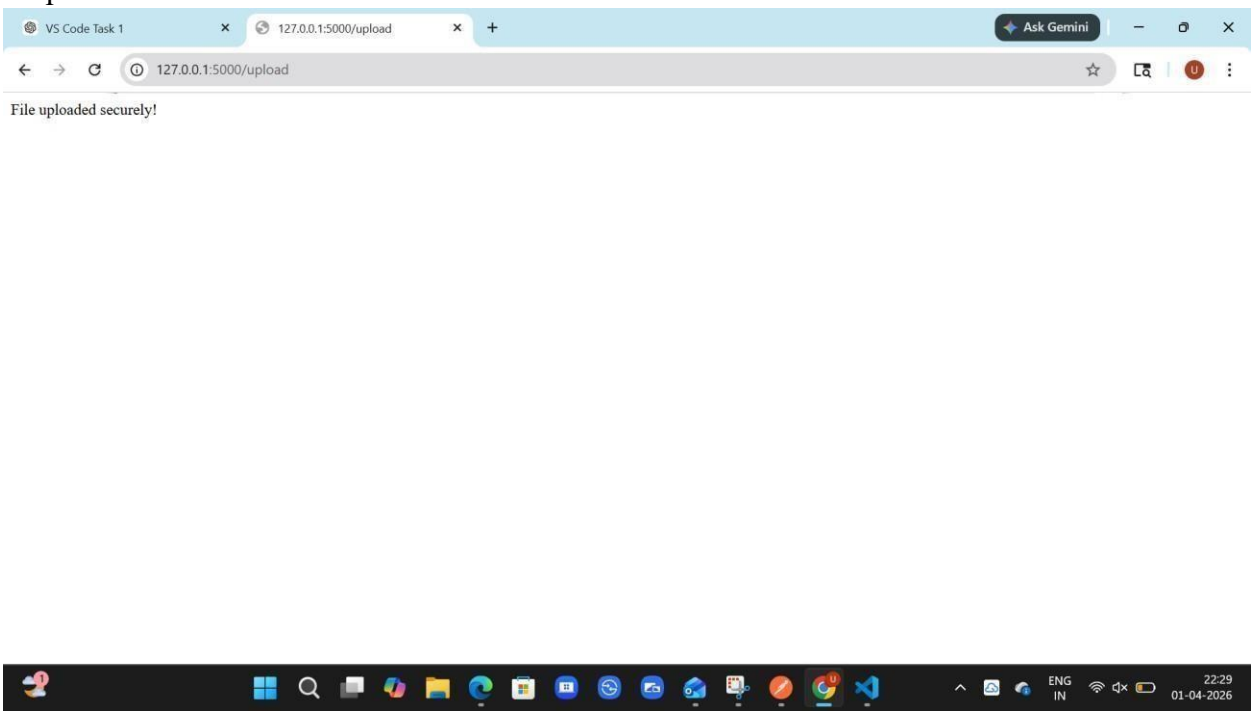
File uploaded successfully!

- The program allows uploading any type of file
- No validation of file extension

Corrected code:



Output:



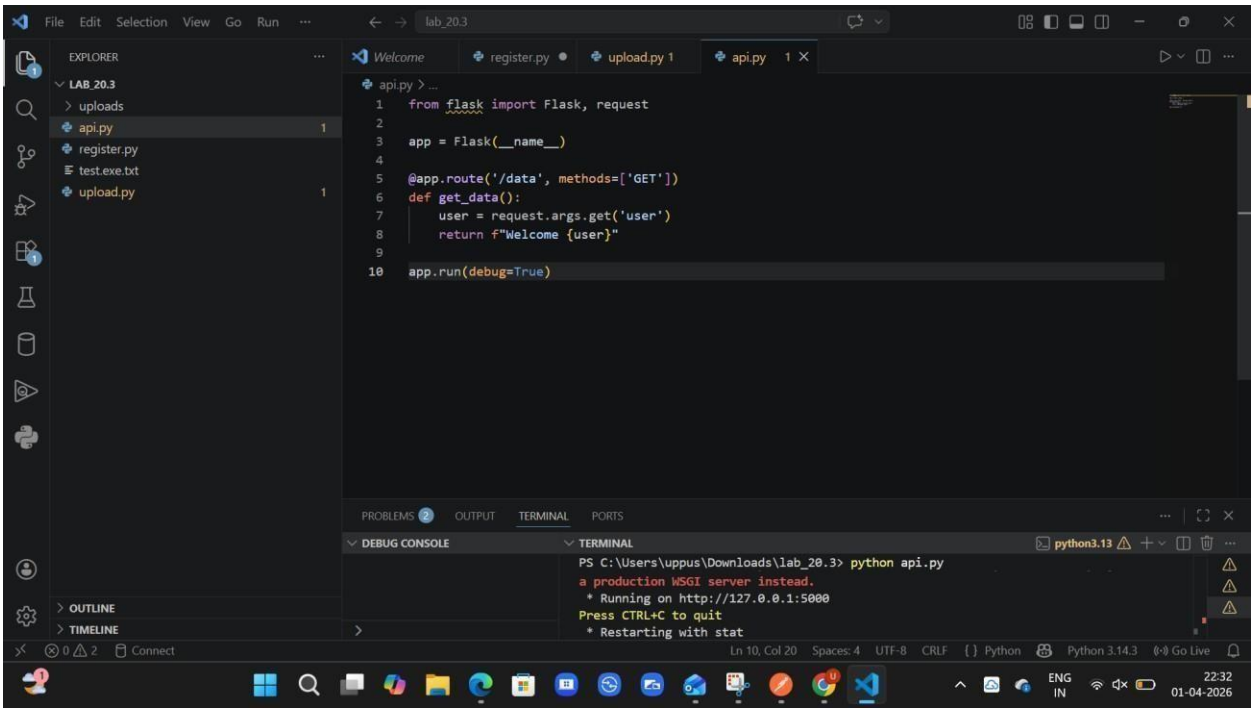
Explanation:

- The initial code allows uploading any file without checking its type, making it insecure.
- Malicious files like `.exe` can be uploaded, which may harm the system.

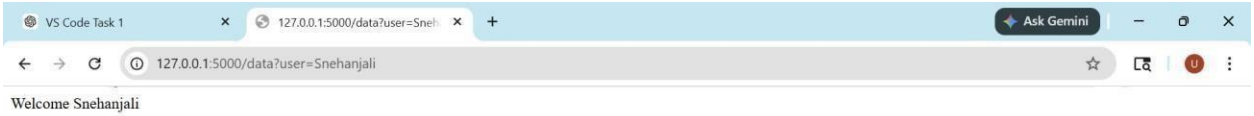
- The improved code restricts file types and allows only safe formats like `.txt`, `.png`, `.jpg`, and `.pdf`. Task3: API Security Review

Prompt:

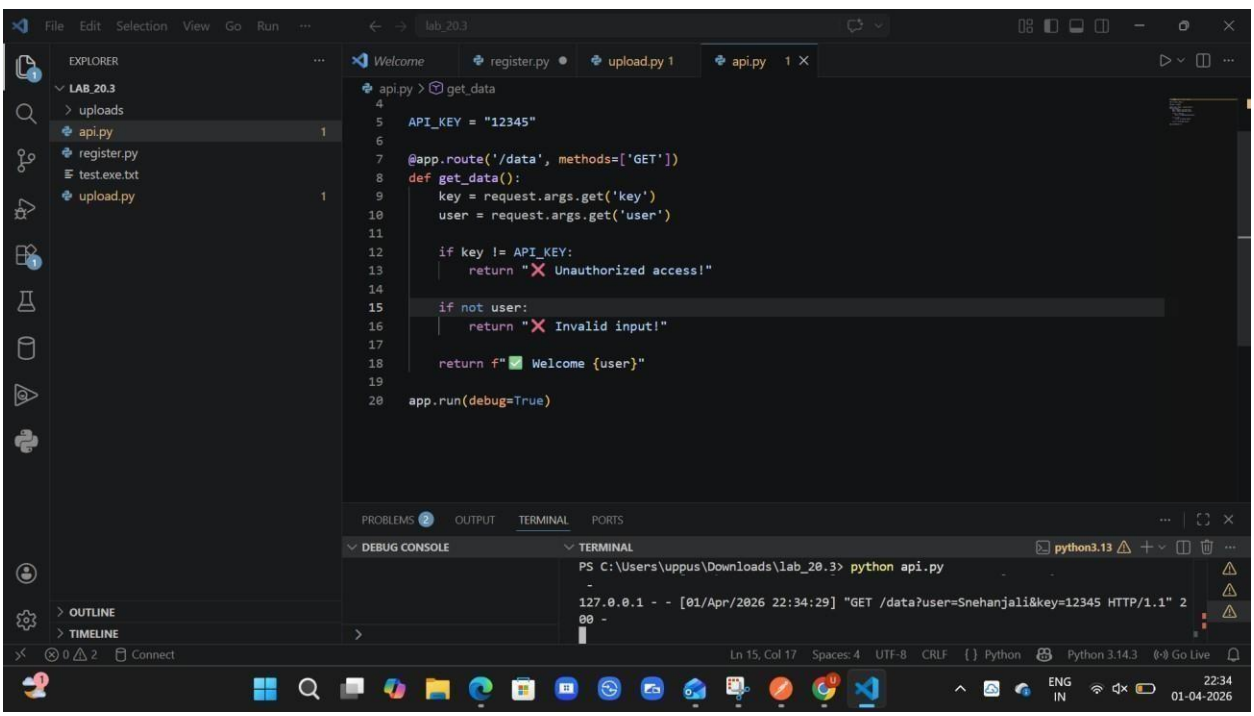
Generate a Python Flask API endpoint for user data Code:



Output:



Corrected code:



Output:

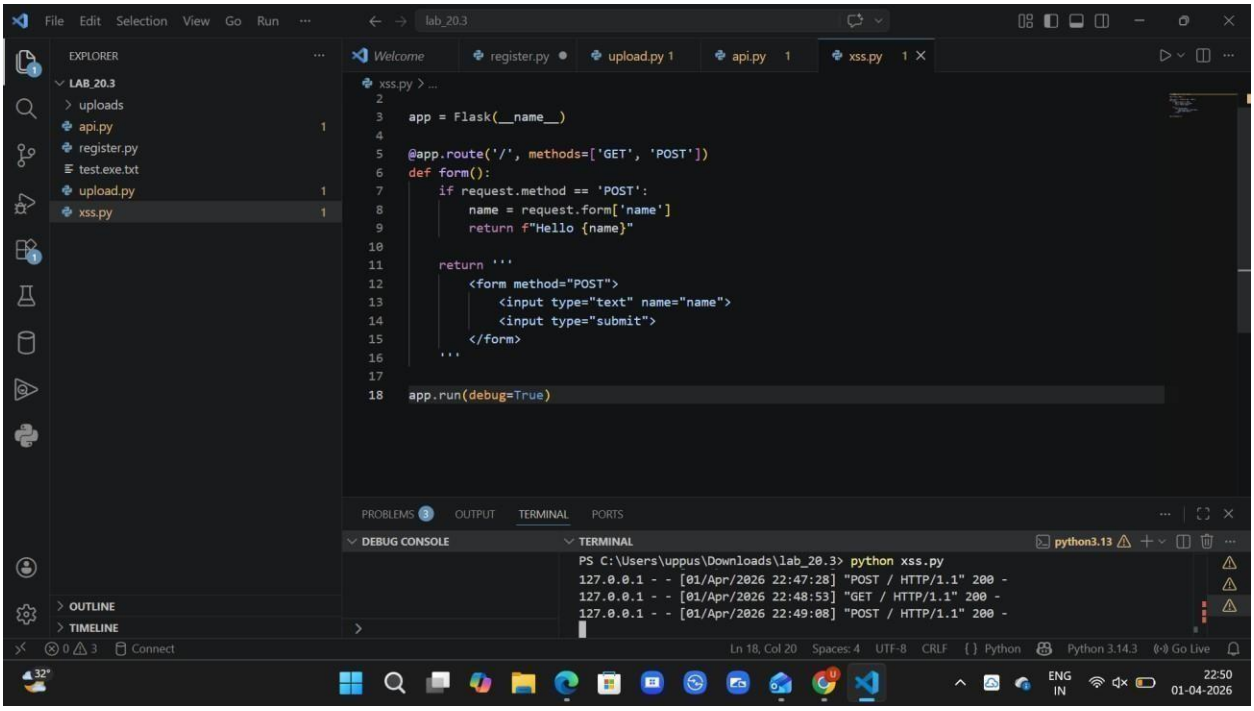
Explanation:

- The initial API does not have authentication, allowing anyone to access it
- It does not validate user input, making it insecure
- The improved API uses an API key and input validation for secure access

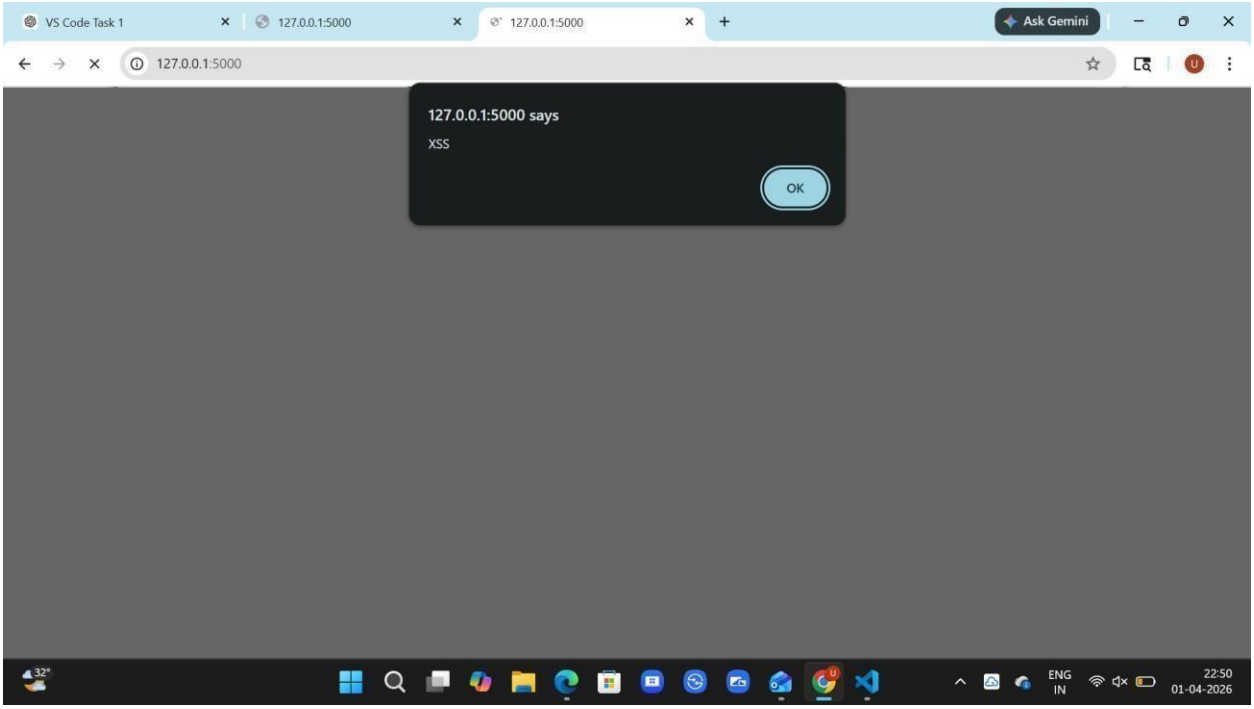
Task 4: Cross-Site Scripting (XSS) Check

Prompt:Generate a feedback form using HTML and JavaScript

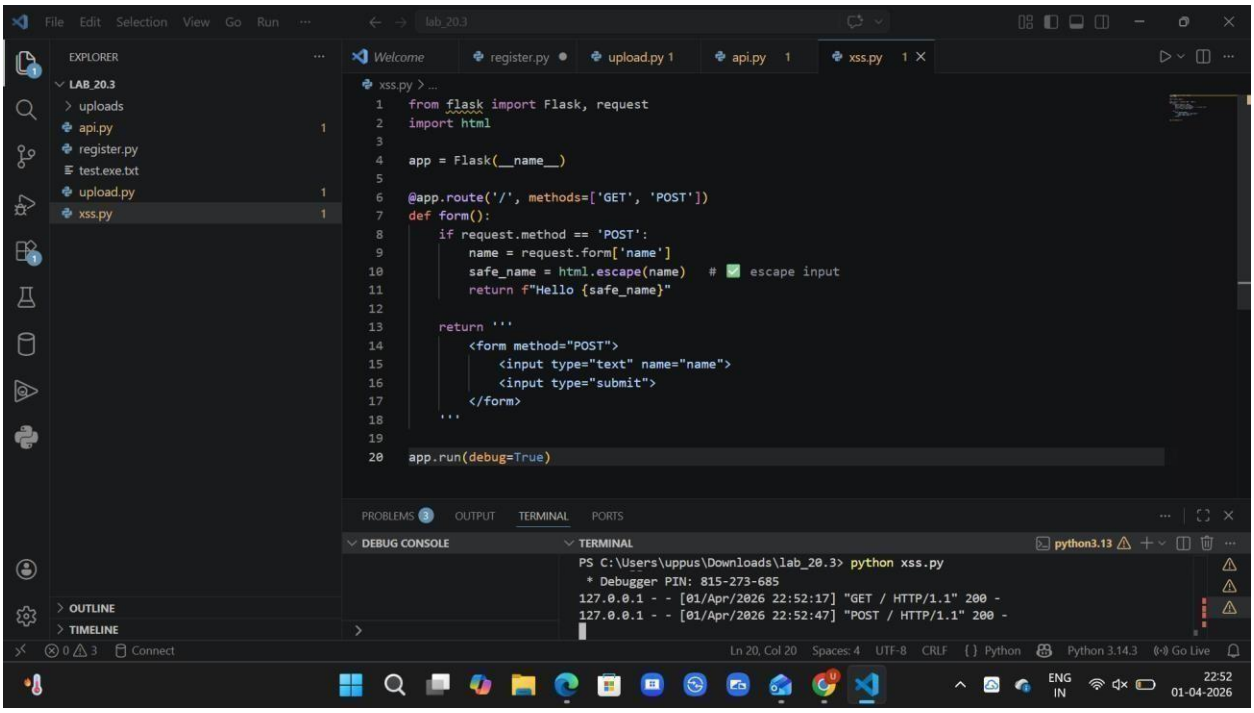
Code:



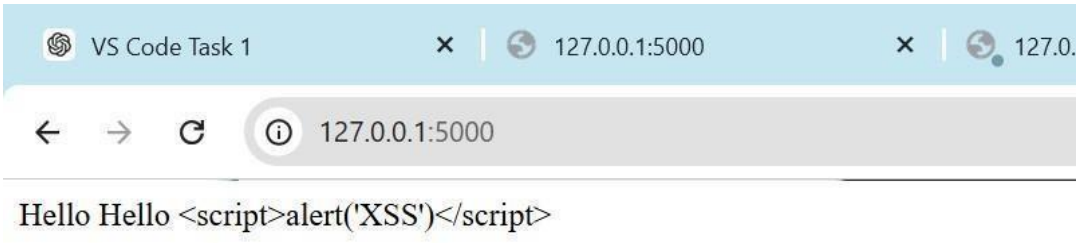
Output:



Corrected Code:



Output:



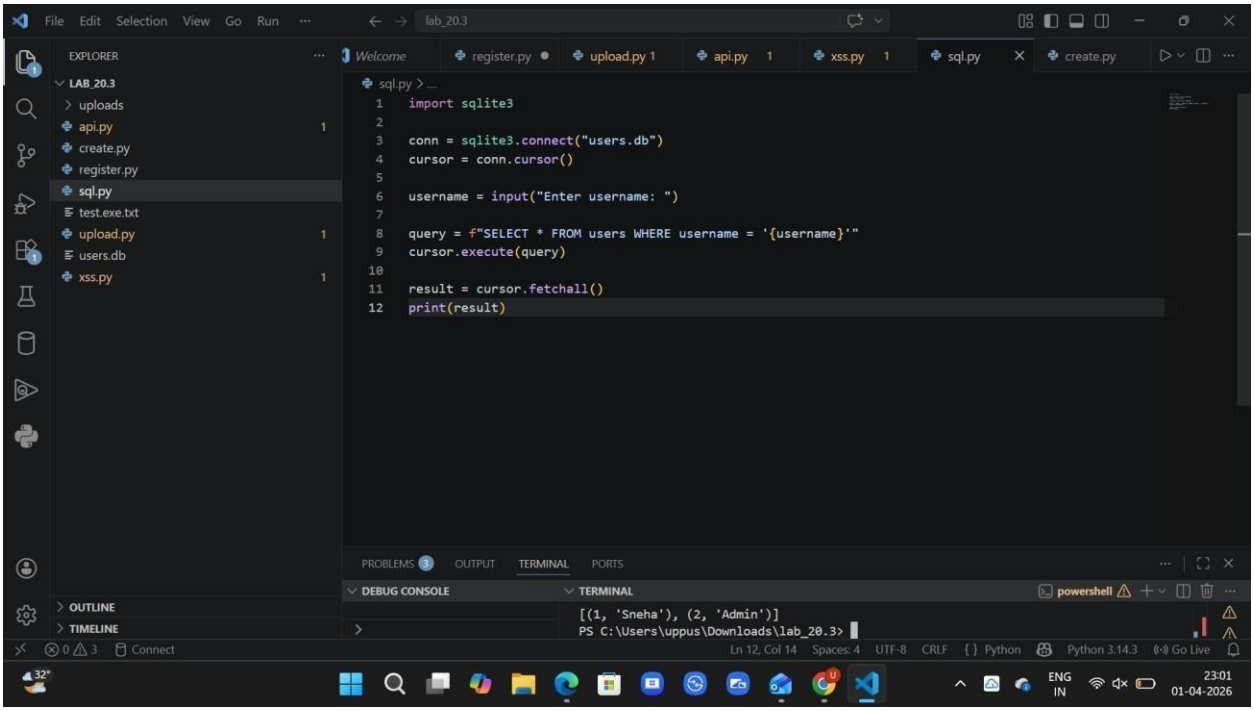
Explanation:

- The initial code displays user input directly without any validation, making it vulnerable to XSS attacks.
- Malicious scripts like `<script>alert('XSS')</script>` can execute in the browser.
- The secure code uses input escaping to prevent script execution and ensures safe output.

Task 5: SQL Injection Prevention

Prompt: Generate a Python script using SQLite to fetch user details from a database.

Code:



Output:

```
PS C:\Users\uppus\Downloads\lab_20.3> python sql.py
Enter username: ' OR '1'='1
[(1, 'Sneha'), (2, 'Admin')]
PS C:\Users\uppus\Downloads\lab_20.3>
```

Corrected code:

```python
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ")

query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))

result = cursor.fetchall()
print(result)
```

Output:

```
PS C:\Users\uppus\Downloads\lab_20.3> python sql.py
Enter username: ' OR '1'='1
[]
PS C:\Users\uppus\Downloads\lab_20.3>
```

Explanation:

- The secure code uses parameterized queries (?) instead of string concatenation.
- User input is treated as data, not as part of the SQL query.
- This prevents SQL Injection and ensures safe database operations.